Instructions

Quick Start

This section is for readers who just want to run the code without following the detailed explanations. Follow these steps in order.

1. Install GAP and its dependencies (Section 1.1).
2. Copy the `Codes` folder from the GitHub repository into your GAP folder (Section 1.2).
3. Install Python and the required packages in a virtual environment (Section 1.3).
4. Load your algebra by creating a `.g` file and a matching `.py` file, or use one of the preloaded algebras (Section 2).
5. Open `Main.g`, set `main_path` to the correct directory, and make sure the Python path inside `Exec(...)` matches your virtual environment (Sections 2.1–2.2).
6. Run `Main.g` in GAP. Output appears in a text file named `cohomology` (Section 2.3).

1 Installation

1.1 Installing GAP

Open a terminal in Ubuntu and navigate to the directory where you want to install GAP. Then run the following commands.

1. Install dependencies

Action:

```
sudo apt update
sudo apt install build-essential gcc g++ make m4 \
    zlib1g-dev libgmp-dev wget
```

2. Download and extract GAP

Action:

```
wget https://github.com/gap-system/gap/releases/download/\
v4.14.0/gap-4.14.0.tar.gz
tar -xzf gap-4.14.0.tar.gz
cd gap-4.14.0
```

Action: Copy the `gap` file provided in the GitHub repository into the `gap-4.14.0` folder.

1.2 Copying the Code Files

Action: Inside the folder where GAP is installed, create a new folder (you may name it anything, e.g., `Codes`), and copy all the `.g` and `.py` files from the `Codes` folder in the GitHub repository into it.

1.3 Installing Python

Open a separate terminal and follow the steps below.
1. Install Python and virtual environment support

Action:

```
sudo apt install python3
sudo apt install python3-venv
```

2. Create and activate a virtual environment

Action:

```
python3 -m venv venv
source venv/bin/activate
```

3. Install required packages

Action:

```
pip install numpy sympy
```

You can close this terminal once the installation is complete.

Action: Copy `pathalg.gi` from the GitHub repository. In your GAP folder, navigate to `pkg`, then locate the `qpa` folder inside it, and within that the `lib` folder. Replace the existing `pathalg.gi` in that `lib` folder with the one from the repository.

The default functions in the original `pathalg.gi` include steps such as computing a basis, which is time-consuming and not required for our code to run. The replacement file removes this overhead.

1.4 Starting GAP

Action: Navigate to your GAP folder and start GAP with the following command, which ensures the QPA package (and its modified files) are loaded correctly:

```
./gap -l "./pkg;."
```

Once GAP has started, you are ready to run `Main.g` as described below.

2 Loading the Algebra

Once everything is installed, we are ready to dive into the computation. We start by loading the algebra, for which you need to create two files: a `.g` file and a `.py` file. Give them the same name to avoid confusion, or simply load your algebra into one of the existing file pairs, e.g., `Algebra.g` with its corresponding `allrels.py`, or `A-ZAlgebras.g` with its corresponding `A-ZAlgebras.py`. There are approximately 68–73 preloaded algebras along with their relations. The format for loading algebras is self-explanatory and can be inferred directly from the existing code.

Action: Make sure the variable `type` in the `.g` file and `typ` in the corresponding Python file are set to the same name.

Once the algebra is loaded (with relations), we are ready to run the code.

2.1 Main.g

Open the `Main.g` file. Here we call the function that creates the quiver. In the code, you will see:

```
Read("Codes/ALGEBRAS.g");
```

This is where the quotient of the path algebra gets loaded.

Action: Set `main_path` to the correct path according to your directory structure:

```
main_path := "/gap-4.14.0/Codes/";
```

This is the path inside the GAP folder where your code files reside, and it is also where the results will be stored.

```
mod_spec := ComputeModuleSpec( A, type, main_path );
```

This function computes the action of the module, allowing you to construct a module over the path algebra. Here, `A` is the algebra loaded from `Algebras.g`, and `type` is also specified in the `.g` file. You will find a folder named `"type"`, containing a `mod_spec` file inside it.

In principle, the whole process could be carried out using the single built-in function `AlgebraAsModuleOverEnvelopingAlgebra(A)`. However, this function only works when the algebra is simple, in the sense that it has no relations. Since our algebras involve substantial relations, this built-in function does not directly apply. We therefore examined the source code of `AlgebraAsModuleOverEnvelopingAlgebra` and modified it to work correctly for algebras with relations.

The output of `mod_spec` is not directly in the form we want for the next stage of the computation. We require the data in the form of vectors, which is achieved using Python. This modified function serves the role of the built-in `Coefficients` function, but for algebras with relations. A sample output for an algebra can be found on GitHub.

2.2 Python Inside GAP

Action: Open `Coefficients.py`. Near the top, you will see the line:

```
from AllRels import rels, typ
```

Here, `AllRels` refers to the `.py` file where you loaded your algebra. If you loaded your algebra in a different file, replace `AllRels` with the name of that file. Also, near the end of the same file, there are a few paths specified — make sure these are consistent with the paths you are using elsewhere in the setup.

Action: Make sure the path below points to your Python virtual environment and to the location of `Coefficients.py` on your system:

```
Exec("/myenv/bin/python3 /gap-4.14.0/Codes/Coefficients.py");
```

This runs `Coefficients.py` in Python, which takes `mod_spec` as input (from the step above) and returns the corresponding map in the form of vectors. This is required for GAP to run `RightModuleOverPathAlgebra` to construct a module, and subsequently form the projective resolution.

2.3 Final Output

The steps above are all carried out within `Main.g`.

Action: Once you have set the input algebra and the correct path names, run `Main.g` in your terminal.

This produces a text file named `cohomology`, containing the dimensions of the Hochschild cohomology in degrees 1, 2, and 3, along with the time taken to run the code. Depending on the algebra, this typically takes between 10 and 120 minutes.

3 Summary of the Workflow